

FORWARD DEPLOYMENT



ENGINEER HANDBOOK

From Customer Problems
to Production Systems



UNDERSTAND
CUSTOMERS



DESIGN
SOLUTIONS



BUILD & INTEGRATE
SYSTEMS



DEPLOY TO
PRODUCTION



OBSERVE &
IMPROVE

- ✓ Problem Discovery
- ✓ System Design
- ✓ Integration & APIs
- ✓ Cloud & Kubernetes
- ✓ CI/CD & Deployment

- ✓ Observability & Debugging
- ✓ Security & Best Practices
- ✓ Incident Response
- ✓ Customer Outcomes
- ✓ Production Excellence



**BUILD. DEPLOY. SOLVE.
DELIVER REAL OUTCOMES.**

ENGINEERING SOLUTIONS THAT DRIVE CUSTOMER SUCCESS



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

FORWARD DEPLOYMENT ENGINEERING EXPLAINED

UNDERSTAND THE ROLE. OWN THE OUTCOME.



WHAT IS A FORWARD DEPLOYED ENGINEER?

A Forward Deployed Engineer (FDE) is an engineer who works directly with customers to understand their problems, design solutions, build integrations, deploy software, and ensure those solutions succeed in production.



WHY COMPANIES USE FDES

- Customers have unique, complex needs
- Bridge the gap between product capabilities and real-world use
- Accelerate time-to-value
- Reduce risk during deployment
- Improve customer success and retention
- Provide deep technical feedback to improve the product

FDE VS OTHER ROLES

ROLE	FORWARD DEPLOYED ENGINEER	SOFTWARE ENGINEER	SOLUTIONS ENGINEER	DEVOPS / PLATFORM ENGINEER
 PRIMARY FOCUS	Customer problems and production outcomes	Build and improve product features	Design solutions and demo value	Build and operate infrastructure and platforms
 WHERE THEY WORK	On-site / Remote with customers	At product organization	Pre-sales and engagements	Internal IT / Platform teams
 WHAT THEY DELIVER	Working solutions in customer environments	Product code and capabilities	Solution designs and architectures	Reliable, scalable infrastructure
 TIME HORIZON	Short to medium term (deploy now, fix now)	Medium to long term (build for the future)	Short term (win the deal)	Long term (operate forever)
 SUCCESS MEASURE	Customer outcome and adoption	Code quality and features	Deals won and value shown	Uptime, reliability, and efficiency

THE CORE MENTAL MODEL



JUNIOR ENGINEER TIP

FDE is engineering, not just technical support. You are solving real problems, building real solutions, and owning the production result. Think like an engineer. Act like an owner.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

THE FDE PRODUCTION MINDSET

THINK LIKE AN ENGINEER. SOLVE REAL PROBLEMS. OWN THE OUTCOME.

1 START WITH THE PROBLEM, NOT THE TECHNOLOGY



- Understand the real problem
- Focus on why it matters
- Avoid jumping to solutions
- Define success before design

5 MOVE QUICKLY WITHOUT BECOMING CARELESS



- Deliver value in small steps
- Iterate and improve
- Don't compromise on quality
- Speed with discipline

2 UNDERSTAND USERS AND WORKFLOWS



- Talk to users
- Observe how they work
- Map their workflows
- Build what actually helps

6 OWN IMPLEMENTATION AND VERIFICATION



- You ship it, you own it
- Verify in every environment
- Don't hand off unverified work
- Be accountable for outcomes

3 WORK WITH INCOMPLETE REQUIREMENTS



- Requirements will change
- Ask the right questions
- Make assumptions explicit
- Validate early and often

7 FEEDBACK LOOPS



- Get feedback early
- Learn from customers
- Improve the solution
- Repeat the cycle

4 MAKE TECHNICAL TRADEOFFS



- Balance speed, quality, cost, and complexity
- Choose good enough, then improve
- Every decision has a cost

8 PRODUCTION OWNERSHIP



- Monitor your solution
- Respond to issues
- Learn from failures
- Drive long-term reliability



THE FDE MINDSET IN ONE LINE

Solve the right problem, build the right solution, ship it safely, and own the outcome.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

REAL PROBLEMS. REAL SOLUTIONS. REAL IMPACT.

DISCOVERING THE REAL CUSTOMER PROBLEM

FIND THE RIGHT PROBLEM. SOLVE THE RIGHT PROBLEM.

1 REQUIREMENTS GATHERING



- Understand the business goal
- Identify stakeholders
- Document pain points
- Clarify the desired outcome
- Validate assumptions early

5 EXISTING ARCHITECTURE



- What systems already exist?
- How do they interact?
- Any integrations in place?
- Known limitations or bottlenecks?
- Avoid duplicating or breaking what already works

2 ASKING TECHNICAL QUESTIONS



- What are you trying to achieve?
- What happens today?
- What is not working?
- What tools and systems do you currently use?
- What are your constraints?
- How will success be measured?

6 SECURITY REQUIREMENTS



- Authentication & authorization
- Data protection
- Encryption requirements
- Compliance standards (GDPR, HIPAA, etc.)
- Access control and least privilege
- Audit and logging requirements

3 FUNCTIONAL vs NON-FUNCTIONAL REQUIREMENTS



FUNCTIONAL (What the system does)

- Features
- User actions
- Business rules
- Workflows

NON-FUNCTIONAL (How the system performs and behaves)

- Performance
- Scalability
- Availability
- Reliability
- Usability
- Maintainability
- Compliance

7 DATA REQUIREMENTS



- What data is used or generated?
- Data sources
- Data quality expectations
- Data storage and retention
- Privacy and ownership
- Data flows and transformations

4 CONSTRAINTS



- Budget
- Timeline
- Team skills
- Technology limitations
- Third-party limitations
- Compliance and regulations
- Operational constraints

8 SUCCESS CRITERIA



- What does success look like?
- How will you measure success?
- KPIs and metrics
- Business outcomes
- User satisfaction
- Technical outcomes

TURNING "WE NEED AI/CLOUD/KUBERNETES" INTO AN ACTUAL ENGINEERING PROBLEM



VAGUE REQUEST
"We need AI/cloud/Kubernetes"



ASK WHY
Uncover the real business problem



DEFINE THE PROBLEM
Clear, specific, measurable problem



DESIGN SOLUTION
Choose the right tools and approach



DELIVER OUTCOME
Solve the problem. Create value.

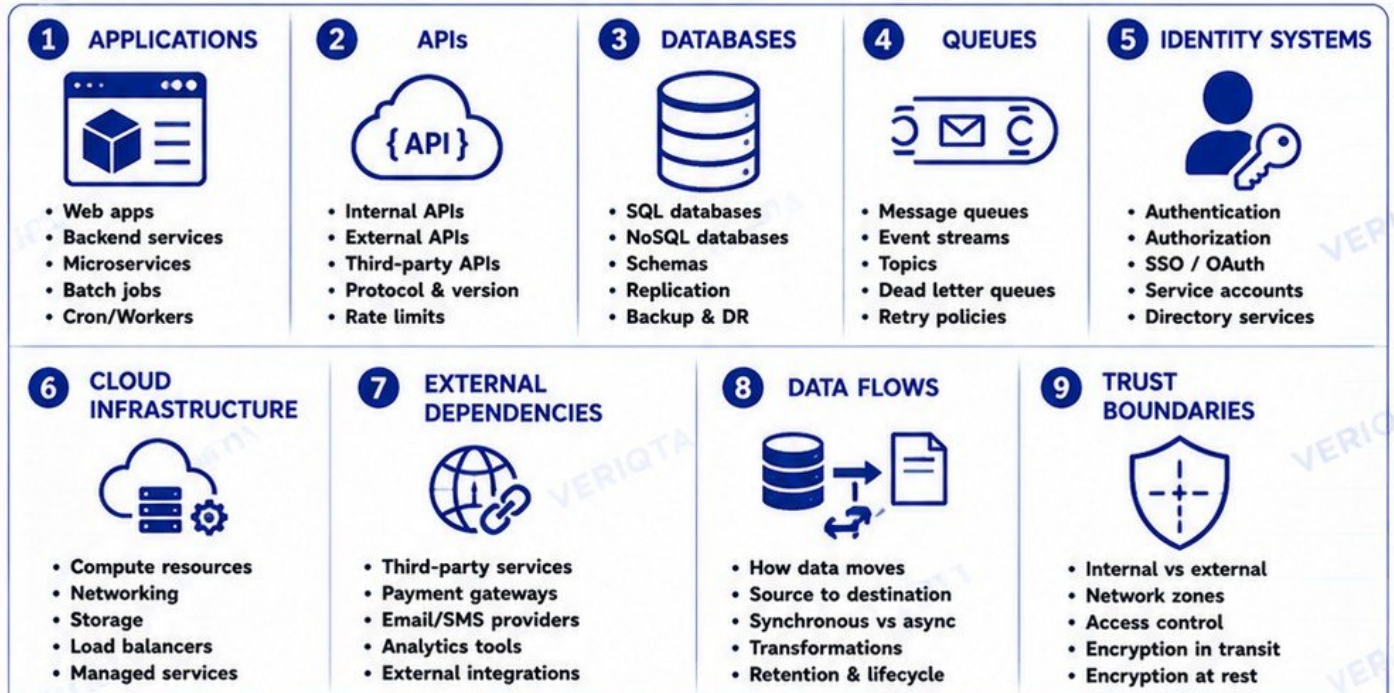


JUNIOR ENGINEER TIP

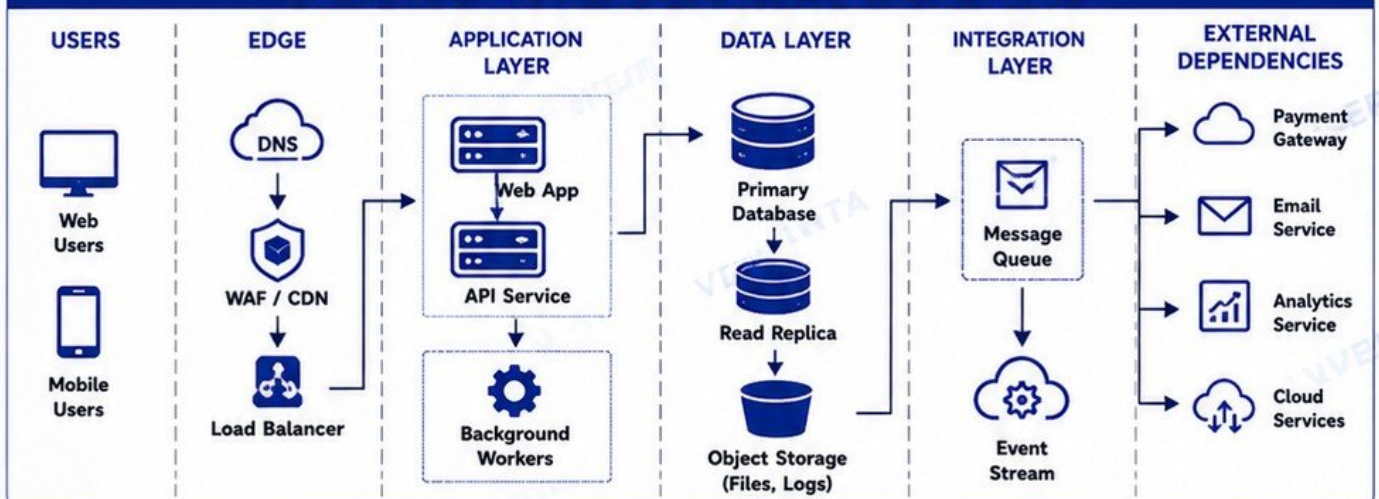
Don't accept a solution before you understand the real problem.
Great FDEs ask better questions, not just build solutions.

MAPPING AN EXISTING PRODUCTION SYSTEM

UNDERSTAND BEFORE YOU CHANGE. MAP BEFORE YOU BUILD.



EXAMPLE: SIMPLE PRODUCTION ARCHITECTURE MAP



CREATE A SIMPLE ARCHITECTURE MAP BEFORE YOU CHANGE ANYTHING.
YOU CAN'T IMPROVE WHAT YOU DON'T UNDERSTAND.



JUNIOR ENGINEER TIP
Diagram the system.
Ask questions. Validate.
Don't assume anything.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

APIs, HTTP, AND SYSTEM INTEGRATION

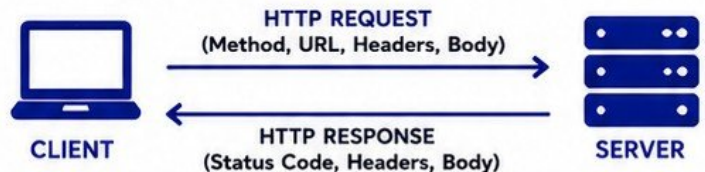
APIs ARE THE LANGUAGE OF MODERN SYSTEMS.
FDEs CONNECT, INTEGRATE, AND DELIVER VALUE THROUGH APIS.



WHY FDES WORK HEAVILY WITH APIS

- Integrate with customer systems
- Extend product capabilities
- Automate workflows
- Exchange data in real time
- Solve problems across systems

HTTP REQUESTS AND RESPONSES



HTTP METHODS

GET	Retrieve data from a resource
POST	Create a new resource
PUT	Replace an entire resource
PATCH	Partially update a resource
DELETE	Delete a resource

HEADERS



- Provide metadata
- Control behavior
- Examples:
 - Content-Type
 - Accept
 - User-Agent
 - Authorization
 - X-Request-ID

AUTHENTICATION



- API Key
- Basic Auth
- Bearer Token (OAuth 2.0)
- JWT (JSON Web Token)
- mTLS / Client Certificates
- Always use least privilege
- Never hardcode credentials

JSON (DATA FORMAT)



- Lightweight and readable
- Language independent
- Common for APIs
- Example:

```
{
  "id": 123,
  "name": "Order",
  "status": "shipped",
  "amount": 199.99
}
```

STATUS CODES

2xx

Success
200 OK, 201 Created

3xx

Redirection
301 Moved, 304 Not Modified

4xx

Client Errors
400 Bad Request, 401 Unauthorized,
403 Forbidden, 404 Not Found,
429 Too Many Requests

5xx

Server Errors
500 Internal Server Error,
502 Bad Gateway, 503 Service Unavailable



TIMEOUTS

- Prevent hanging requests
- Set connection and read timeouts
- Fail fast, don't wait forever



RETRIES

- Handle transient failures
- Use exponential backoff
- Add jitter to avoid thundering herd



RATE LIMITS

- APIs have limits
- Respect limits to avoid being throttled or blocked
- Handle 429 responses gracefully

TESTING APIS WITH CURL



```
# GET request
curl -X GET https://api.example.com/v1/users

# POST request with JSON body
curl -X POST https://api.example.com/v1/users \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <token>" \
-d '{"name": "John", "email": "john@example.com"}'
```

```
# PUT request
curl -X PUT https://api.example.com/v1/users/123 \
-H "Content-Type: application/json" \
-d '{"name": "John Updated"}'

# Add headers and see response details
curl -i -X GET https://api.example.com/v1/orders \
-H "Accept: application/json"
```



JUNIOR ENGINEER TIP

Understand the API contract. Handle errors. Validate data.
Good integrations make great engineers.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

WORKING WITH DATA IN CUSTOMER ENVIRONMENTS

GOOD DATA IN. GOOD INSIGHTS OUT.

1 STRUCTURED VS UNSTRUCTURED DATA



STRUCTURED DATA

- Organized in rows/columns
- Easy to store, query
- Examples: SQL tables, CSV, JSON



UNSTRUCTURED DATA

- No predefined format
- Harder to search and analyze
- Examples: logs, documents, images, emails

2 JSON AND CSV

JSON (JavaScript Object Notation)



```
{  "id": 101,  "name": "Alice",  "email": "alice@example.com",  "orders": [    { "id": 1, "total": 49.99 }  ]}
```

CSV (Comma Separated Values)



```
id,name,email,total
101,Alice,alice@example.com,49.99
102,Bob,bob@example.com,19.99
103,Carol,carol@example.com,29.99
```

3 DATABASES



- Store, organize and protect data
- Types: Relational (SQL), NoSQL, Time-series, etc.
- Choose the right database for the right use case
- Know connection methods, drivers, and limitations

4 SQL FUNDAMENTALS



- Query and manipulate relational data
- Key SQL commands:

```
SELECT * FROM users;

SELECT id, name FROM users WHERE status = 'active';

INSERT INTO users (name, email) VALUES ('John', 'john@example.com');

UPDATE users SET status = 'inactive' WHERE id = 101;

DELETE FROM users WHERE id = 101;
```

- Understand joins, filtering, grouping, ordering
- Always test queries safely

5 DATA TRANSFORMATION



- Clean and reshape data to fit your needs
- Common operations:
 - Filter
 - Map / Rename
 - Aggregate
 - Normalize / Denormalize
 - Enrich / Join



RAW DATA



TRANSFORM



CLEAN DATA

6 SCHEMA MISMATCHES



- Different systems, different schemas
- Field names may differ
- Data types may differ
- Missing fields
- Extra fields
- Strategy:
 - Map fields
 - Use adapters
 - Validate and transform

7 DATA VALIDATION



- Validate early, validate often
- Check:
 - Data types
 - Formats
 - Ranges
 - Required fields
 - Business rules
- Reject bad data fast

8 MISSING AND MALFORMED DATA



- Data is never perfect
- Handle gracefully
- Strategies:
 - Default values
 - Skip or quarantine
 - Error logs & alerts
 - Communicate issues

9 PII AWARENESS



- PII = Personally Identifiable Information
- Examples: name, email, phone, address, SSN, ID numbers, etc.
- Protect it:
 - Encrypt
 - Limit access
 - Mask in logs
 - Comply with laws (GDPR, CCPA, etc.)

10 NEVER TRUST INCOMING DATA BLINDLY



- Attackers and bugs send bad data
- Validate everything
- Sanitize inputs
- Use least privilege
- Log suspicious activity
- Assume nothing. Verify everything.



JUNIOR ENGINEER TIP

Understand your data. Clean it. Validate it. Protect it.
Good data practices prevent bad systems.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta



LINUX AND THE COMMAND LINE FOR FDES



ESSENTIAL TERMINAL SKILLS TO DEPLOY, DEBUG, AND DELIVER SOLUTIONS



1. NAVIGATING SERVERS

Move around the filesystem.

<code>pwd</code>	Show current directory
<code>ls -lah</code>	List files (all details)
<code>cd /path/to/dir</code>	Change directory
<code>cd ..</code>	Go up one level
<code>cd ~</code>	Go to home directory



2. FILES & PERMISSIONS

View and manage files safely.

<code>touch file.txt</code>	Create a file
<code>cp src.txt dest.txt</code>	Copy file
<code>mv old.txt new.txt</code>	Move/rename file
<code>rm file.txt</code>	Remove file
<code>chmod 644 file.txt</code>	Set permissions
<code>chown user:group file.txt</code>	Change owner



3. PROCESSES

Inspect running processes.

<code>ps aux</code>	All running processes
<code>ps -ef grep app</code>	Filter process
<code>top</code>	Live process monitor
<code>kill PID</code>	Stop a process
<code>kill -9 PID</code>	Force stop a process



4. ENVIRONMENT VARIABLES

View and manage environment.

<code>printenv</code>	Show all variables
<code>echo \$PATH</code>	Show PATH value
<code>echo \$APP_ENV</code>	Show variable value
<code>export VAR=value</code>	Set variable (temporary)



5. NETWORKING COMMANDS

Check connectivity and ports.

<code>ip a</code>	Show IP addresses
<code>ss -tuln</code>	List listening ports
<code>ping host</code>	Check connectivity
<code>curl -I https://example.com</code>	Check HTTP response
<code>nslookup domain.com</code>	DNS lookup



6. LOGS

Read and search logs.

<code>cat /var/log/syslog</code>	View log file
<code>less /var/log/app.log</code>	Scroll log
<code>tail -f /var/log/app.log</code>	Follow live logs
<code>grep "error" /var/log/app.log</code>	Search for errors



7. SERVICES

Manage system services.

<code>systemctl status nginx</code>	Check service status
<code>systemctl start nginx</code>	Start service
<code>systemctl stop nginx</code>	Stop service
<code>systemctl restart nginx</code>	Restart service
<code>systemctl enable nginx</code>	Start on boot

IMPORTANT COMMANDS EVERY FDE USES



8. SSH

Securely connect to servers.

<code>ssh user@host</code>	Connect to server
<code>ssh -i key.pem user@host</code>	With private key
<code>ssh-copy-id user@host</code>	Copy your SSH key



9. CURL

Make HTTP requests.

<code>curl https://api.example.com</code>	GET request
<code>curl -X POST -d '{}' https://api.example.com</code>	POST request
<code>curl -H "Authorization: Bearer TOKEN" https://api.example.com</code>	With auth header



10. GREP

Search text or logs.

<code>grep "error" file.log</code>	Search in file
<code>ps aux grep app</code>	Search in command output
<code>grep -i "failed" file.log</code>	Case-insensitive search



11. PS

Quick process snapshot.

<code>ps aux head</code>	Top processes
<code>ps -ef grep app</code>	Find specific process



12. SS

Modern netstat.

<code>ss -tuln</code>	Listening ports
<code>ss -tulpn grep :80</code>	Find process on port 80



13. SYSTEMCTL

Control system services.

<code>systemctl status app</code>	Check status
<code>systemctl restart app</code>	Restart service
<code>systemctl reload app</code>	Reload config (if supported)



14. JOURNALCTL

View systemd logs.

<code>journalctl -u app</code>	Logs for a service
<code>journalctl -u app -f</code>	Follow live logs
<code>journalctl -xe</code>	Recent errors



7. FIND ROOT CAUSE

Confirm evidence.
Identify the cause,
not just the symptom.

15. DIAGNOSING A FAILING APPLICATION FROM THE TERMINAL



1. IDENTIFY SYMPTOM

What is failing?
Errors? Timeouts?
Unreachable?



2. CHECK SERVICE

`systemctl status app`
Is the service up?
Any recent failures?



3. CHECK LOGS

`journalctl -u app -xe`
Look for errors
and stack traces.



4. CHECK PROCESSES

`ps aux | grep app`
Is the process running?
High CPU or memory?



5. CHECK NETWORK

`ss -tuln`
Is app listening?
Any port issues?



6. TEST ENDPOINTS

`curl http://localhost:PORT`
Test locally first.
Check API/health.



7. FIND ROOT CAUSE

Confirm evidence.
Identify the cause,
not just the symptom.



8. APPLY FIX & VERIFY

Fix the issue.
Re-test and
monitor logs.



JUNIOR ENGINEER TIP

The terminal is your best debugging tool.
Read logs. Ask questions. Follow the evidence.
Never guess—verify.



REMEMBER

Great FDEs solve real problems fast.
The command line gives you clarity, control,
and confidence in any environment.

GIT AND COLLABORATIVE DELIVERY

WORK TOGETHER. SHIP FASTER. KEEP EVERY CHANGE TRACEABLE.

1 REPOSITORIES



A repository (repo) stores your project's entire history.

```
git init
```

```
git clone <url>
```

Initialize or clone a repo

2 BRANCHES



Branches let you work on features without affecting main code.

```
git branch
git branch <name>
git checkout <name>
or git switch <name>
```

List, create and switch branches

3 COMMITS



Commits capture changes with a message and become part of history.

```
git add .
git commit -m "feat: add login API"
git log --oneline
```

Stage, commit and view history

4 PULL REQUESTS



A pull request (PR) proposes your changes for review before merging.

```
git push -u origin <branch>
```

Open PR on your Git platform (GitHub, GitLab, Bitbucket)

5 CODE REVIEW



Reviews improve quality, catch issues early and share knowledge.

- Check correctness
- Check security
- Check performance
- Check style & standards

6 MERGE CONFLICTS



Happens when changes in the same file conflict.

Resolve like a pro:

1. `git status`
2. Open the file, fix conflicts
3. `git add <file>`
4. `git commit`

Communicate and understand changes before editing.

7 RELEASE BRANCHES



Use release branches to prepare, stabilize and ship production-ready code.

```
git checkout -b release/1.2.0
git push -u origin release/1.2.0
```

Release branches help with hotfixes and traceability.

8 CUSTOMER-SPECIFIC CONFIGURATION



Use configuration, not code changes, for customer customization.

- Use env files
- Use config maps
- Use feature flags
- Use overrides

Keep one codebase. Make it configurable.

9 AVOID DANGEROUS CUSTOMER FORKS



Forks create long-term maintenance, security and upgrade headaches.

Instead:

- ✓ Use configuration
- ✓ Use extension points
- ✓ Contribute upstream
- ✓ Keep your repo clean

One codebase. Many customers. Sustainable support.

10 KEEP CHANGES TRACEABLE



Trace every change from code to production.

- Meaningful commit messages
- Link PRs to issues
- Tag every release
- Record deployment info
- Use signed commits (optional)

TYPICAL GIT WORKFLOW



CREATE BRANCH
`git checkout -b feature/xyz`



MAKE CHANGES
Edit, build, test locally



COMMIT
`git add .`
`git commit -m "your message"`



PUSH
`git push -u origin feature/xyz`



OPEN PR
Request review and discuss



REVIEW & APPROVE
Address feedback and get approval



MERGE
Merge to main or release branch



DEPLOY
Changes flow to environments

BEST PRACTICES FOR FDES

- ✓ Commit small, logical changes often.
- ✓ Write clear commit messages (what & why).
- ✓ Pull latest changes before starting work.
- ✓ Communicate in PRs. Explain the why, not just the what.
- ✓ Test before pushing.
- ✓ Keep main branch deployable.
- ✓ Use release branches for stability.
- ✓ Avoid force push on shared branches.
- ✓ Trace every deployed version back to a commit.
- ✓ Document customer-specific configuration.



JUNIOR ENGINEER TIP

Git is not just a tool. It is how teams build, collaborate and deliver safely. Good Git hygiene today prevents production incidents tomorrow.

CONTAINERS AND PORTABLE DEPLOYMENTS



BUILD ONCE. RUN ANYWHERE. DELIVER RELIABLY.



1. WHY FDES USE CONTAINERS

- ✓ Consistency across environments
- ✓ Eliminate "works on my machine"
- ✓ Fast, lightweight and portable
- ✓ Easy to version, ship and roll back
- ✓ Ideal for microservices and integrations
- ✓ Run anywhere: laptop, server, cloud, Kubernetes
- ✓ Speed up delivery and improve reliability

2. IMAGE VS CONTAINER



IMAGE

- Read-only template
- Built from Dockerfile
- Stored in a registry
- Versioned and immutable
- Example: myapp:1.0

VS



CONTAINER

- Running instance of an image
- Has writable layer
- Has its own process, network, ports and filesystem
- Example: myapp_container



3. DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8080
ENV APP_ENV=production
CMD ["python", "app.py"]
```

Build image:
docker build -t myapp:1.0 .



4. REGISTRIES

Store and distribute container images.

- Docker Hub
- Amazon ECR
- Google Artifact Registry
- Azure Container Registry
- Private registries

Push image:
docker push myregistry.com/
myapp:1.0



5. PORTS

Expose container services on specific ports.

```
docker run -d -p 8080:80 myapp:1.0
```

- 8080 = host port
- 80 = container port
- Use -P to publish random ports
- Use EXPOSE in Dockerfile as documentation



6. VOLUMES

Persist data outside the container.

- Bind mount (host path)

```
-v /data/app:/app/data
```
- Named volume (managed by Docker)

```
-v app_data:/app/data
```
- Use volumes for databases, uploads, logs, cache, etc.
- Volumes survive container restarts and deletions



7. ENVIRONMENT VARIABLES

Configure apps without changing code.

```
-e APP_ENV=production \
-e DB_HOST=db \
-e API_KEY=abc123
```

- Use -e or --env-file
- ENV in Dockerfile sets defaults
- Never bake secrets into images



8. CONTAINER LOGS

Logs are your first signal.

View logs:
docker logs <container_id>

Follow logs:
docker logs -f <container_id>

Previous logs (after restart):
docker logs --tail 100 <container_id>

- Send logs to centralized logging in prod



9. HEALTH CHECKS

Let Docker and orchestrators know if your app is healthy.

```
HEALTHCHECK --interval=30s \
--timeout=3s \
--retries=3 \
CMD curl -f http://localhost:8080/health || exit 1
```

- Docker uses this to mark container healthy/unhealthy
- Essential for orchestrators (K8s, ECS, etc.)
- Helps with auto-restart and load balancer routing



10. DEBUGGING A CONTAINER THAT EXITS IMMEDIATELY

- | | |
|----------------------------|---|
| 1 Check logs | docker logs <container_id> |
| 2 Run interactively | docker run -it --entrypoint sh myapp:1.0 |
| 3 Check command/entrypoint | docker inspect <container_id> grep -i entrypoint |
| 4 Verify env vars | docker exec -it <container_id> env |
| 5 Check file permissions | docker exec -it <container_id> ls -l <path> |
| 6 Test dependencies | docker exec -it <container_id> curl http://service:port |
| 7 Check resource limits | docker stats <container_id> |

COMMON DOCKER COMMANDS FOR FDES



Run container
docker run -d
myapp:1.0



List containers
docker ps -a



Stop container
docker stop
<container_id>



Remove container
docker rm
<container_id>



Exec into container
docker exec -it
<container_id> sh



Inspect container
docker inspect
<container_id>



Remove image
docker rmi
myapp:1.0



List images
docker images



Build image
docker build -t
myapp:1.0 .



Tag image
docker tag myapp:1.0
myrepo/myapp:1.0



Push image
docker push
myrepo/myapp:1.0



Pull image
docker pull
myrepo/myapp:1.0



Prune system
docker system
prune -a



Show logs
docker logs -f
<container_id>



JUNIOR ENGINEER TIP

Containers are powerful, but understanding what's inside them is what makes you effective.



FOCUS ON:

- ✓ Reproducible builds
- ✓ Clear configuration
- ✓ Observability
- ✓ Resources & limits
- ✓ Security
- ✓ Fast debugging

**GREAT CONTAINERS
MAKE GREAT
PRODUCTION SYSTEMS.**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

KUBERNETES FOR FORWARD DEPLOYMENT



DELIVER SCALABLE, RESILIENT, AND PORTABLE APPLICATIONS FOR CUSTOMERS.

1 WHY CUSTOMERS USE KUBERNETES



- Runs anywhere: cloud, on-prem, hybrid
- Scales applications up and down
- Self-healing and automation
- Rolling updates with zero/low downtime
- Service discovery and load balancing
- Better resource utilization
- Strong ecosystem and integrations
- Standard for modern app platforms

2 PODS



- Smallest deployable unit in Kubernetes
- Wraps one or more containers
- Share network & storage
- Ephemeral (can be replaced)

```
kubectl get pods
kubectl describe pod <pod-name>
```

3 DEPLOYMENTS



- Manages ReplicaSets and Pods
- Declarative updates and rollbacks
- Ensures desired state
- Supports rolling updates

```
kubectl get deployments
kubectl rollout status deploy/<name>
```

4 SERVICES



- Stable endpoint for accessing Pods
- Service discovery
- Load balancing across Pods
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName

```
kubectl get svc
```

5 CONFIGMAPS



- Store non-sensitive configuration
- Key-value pairs or files
- Decouples config from container images

```
kubectl create configmap app-config \
--from-file=config.yaml
kubectl get configmaps
```

6 SECRETS



- Store sensitive data: passwords, keys, tokens
- Base64 encoded (not encrypted by default)
- Use Secret providers for encryption

```
kubectl create secret generic db-secret \
--from-literal=password=StrongPass
kubectl get secrets
```

7 INGRESS



- Expose HTTP/HTTPS routes
- Layer 7 routing (path/host based)
- Works with Ingress Controllers (NGINX, Traefik, ALB, etc.)

```
kubectl get ingress
```

8 NAMESPACES



- Virtual clusters within a cluster
- Isolate environments and teams
- Helps with multi-tenancy & permissions

```
kubectl get namespaces
```

9 RESOURCE REQUESTS & LIMITS



- Requests = what a container needs
- Limits = max it can use
- Prevents noisy neighbor problems
- Enables efficient scheduling

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

10 PROBES



- Check container health
- Types:
 - ✓ Liveness: Is container alive?
 - ✓ Readiness: Is container ready?
 - ✓ Startup: Slow-start applications

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
```

11 IMPORTANT KUBECTL COMMANDS

```
kubectl get
List resources

kubectl get pods
kubectl get all -n myns

kubectl describe
Detailed info about a resource
kubectl describe pod <pod>

kubectl logs
View logs from a container
kubectl logs <pod>
-c <container> -f

Helpful Flags
-n <namespace>
-o wide
-w (watch)
--all-namespaces
```

12 FDE TROUBLESHOOTING MENTAL MODEL



- 1 Identify the Symptom
e.g., App not responding
- 2 Check the Pod
kubectl get pods
- 3 Describe the Pod
kubectl describe pod <pod>
- 4 Check Logs
kubectl logs <pod> -c <container>
- 5 Check Dependencies
Services, DB, external APIs
- 6 Check Config & Secrets
ConfigMaps, Secrets, Env Vars
- 7 Fix, Redeploy, Verify
Confirm the issue is resolved

BONUS: COMMON KUBERNETES OBJECTS YOU WILL ENCOUNTER



ReplicaSet
Ensures the desired number of Pods



StatefulSet
For stateful applications (databases, etc.)



DaemonSet
Runs a Pod on all (or some) nodes



Job / CronJob
Run tasks once or on a schedule



PersistentVolume (PV)
Cluster storage resource



PersistentVolumeClaim (PVC)
Request storage from a PV



HPA
Auto scale based on metrics



JUNIOR ENGINEER TIP

Kubernetes is powerful but complex. Start simple, observe everything, and always understand the WHY behind each object you use.



REMEMBER

You don't need to memorize everything. Understand the concepts. Use kubectl. Observe the system. Solve real problems.

CLOUD INFRASTRUCTURE AND DEPLOYMENT ENVIRONMENTS

BUILD IT RIGHT. DEPLOY IT ANYWHERE. RUN IT RELIABLY.



1. COMPUTE

- Run your applications
- VMs, Containers, Serverless
- Scale up/down on demand
- Examples: EC2, GCE, Azure VM, ECS, GKE, AKS, Lambda



2. NETWORKING

- Connect resources securely
- VPC/VNet, Subnets
- Routing, Security Groups
- Firewall, NAT, Peering
- Private & Public IPs



3. STORAGE

- Store data durably
- Block, File, Object storage
- Snapshots & backups
- Examples: EBS, EFS, S3, GCS, Blob Storage



4. DATABASES

- Managed or self-managed DBs
- Relational & NoSQL
- High availability & backups
- Examples: RDS, Cloud SQL, DynamoDB, CosmosDB



5. LOAD BALANCERS

- Distribute traffic across targets
- Improve availability & performance
- Health checks & SSL termination
- Types: Application, Network, Gateway



6. DNS

- Resolve domain names to IPs
- Route traffic to the right place
- Health-based routing
- Examples: Route 53, Cloud DNS, Azure DNS



7. IAM (IDENTITY & ACCESS)

- Control who can access what
- Users, Groups, Roles, Policies
- Least privilege access
- MFA & Permission boundaries
- Audit & Access logs



8. PUBLIC VS PRIVATE RESOURCES

- Public: Accessible from the internet
- Private: Only within your network
- Use private subnets for sensitive workloads
- Expose only what is necessary

9. DEPLOYMENT ENVIRONMENTS



DEVELOPMENT

- For building & testing
- Fast feedback
- Less stable
- Lower cost



STAGING / TEST

- Mirror production
- Pre-production validation
- Performance & security testing



PRODUCTION

- Live environment
- Highly available
- Monitored & secure
- Changes are controlled

10. CLOUD ARCHITECTURE MENTAL MODEL



USERS



DNS



LOAD BALANCER



APP / COMPUTE



DATA SERVICES



NETWORKING



STORAGE



IAM



MONITORING



BACKUP

11. WHY "WORKS ON MY MACHINE" IS NOT DEPLOYMENT SUCCESS



WORKS ON MY MACHINE

- Local setup is different
- No real network latency
- No real security controls
- No real load
- No real failures
- Missing cloud services
- Hidden dependencies



SUCCESS IN PRODUCTION

- Same environment
- Real network & security
- Scalable & resilient
- Monitored & observable
- Tested under load
- Backups & recovery
- Designed for failure

12. COMMON CLOUD ARCHITECTURE PATTERNS



3-TIER ARCHITECTURE

- Web
- App
- Database



MICROSERVICES

- Independent services
- Scalable
- Fault isolation



EVENT-DRIVEN

- Publish/Subscribe
- Queues
- Async processing



SERVERLESS

- Pay per use
- Auto scale
- Less ops

13. KEY CLOUD PRINCIPLES



SCALABILITY

Scale with demand



RELIABILITY

Design for high availability



SECURITY

Secure by default



AUTOMATION

Automate everything



OBSERVABILITY

Monitor, log & alert



COST OPTIMIZATION

Right size & reduce waste



JUNIOR ENGINEER TIP

Understand the building blocks. Know how they connect. Always think about security, reliability, and cost before you deploy.



REMEMBER

A good deployment is not just about code. It's about the environment that runs your code. Design smart. Automate. Observe. Improve.

CONFIGURATION, SECRETS, AND ENVIRONMENT MANAGEMENT

MANAGE SETTINGS SECURELY. DEPLOY CONFIDENTLY.



CONFIGURATION VS CODE

Code defines how the application behaves. Configuration defines how it runs.



ENVIRONMENT VARIABLES

Perfect for small, sensitive, or environment-specific settings.



CONFIGURATION FILES

Use for larger settings, structured data, or complex application configuration.



SECRET MANAGERS

Store and retrieve secrets securely. Never keep secrets in your codebase.

SECRETS AND SENSITIVE DATA EXAMPLES



- API KEYS
- DATABASE PASSWORDS
- SERVICE ACCOUNT KEYS
- TOKENS
- SSH KEYS
- CERTIFICATES
- THIRD-PARTY CREDENTIALS

TREAT SECRETS LIKE CROWN JEWELS



- ✓ Never hardcode secrets in code or configuration files
- ✓ Never commit secrets to Git
- ✓ Never share secrets in chat, tickets, or email
- ✓ Rotate secrets regularly
- ✓ Log access, not values

DEVELOPMENT vs PRODUCTION CONFIGURATION

DEVELOPMENT



- Easy to change
- Local or shared env files
- Mock services allowed
- Debug logging enabled

PRODUCTION



- Secure and locked down
- Secrets from Secret Manager
- Real services and data
- Minimal logging

Never use development settings in production. Small differences can cause big outages.

BEST PRACTICES



AVOID HARDCODED SECRETS

Hardcoded secrets lead to breaches and compliance failures.



LEAST PRIVILEGE

Give only the permissions and data access required. Nothing more.



CUSTOMER ISOLATION

Use separate environments, namespaces, and credentials. Never mix customer data or access.



CONFIGURATION DRIFT

Drift happens when environments diverge over time. Detect it. Document it. Automate it.

COMMON SECRET MANAGERS



AWS Secrets Manager



Google Secret Manager



HashiCorp Vault



Azure Key Vault



Doppler, 1Password, Infisical, etc.

SECURE CONFIGURATION FLOW



CODE WITHOUT SECRETS



CONFIGURATION SOURCES



SECRET MANAGER



INJECT AT RUNTIME



APPLICATION RUNS SECURELY



Secrets stay in the vault. Applications only get what they need, when they need it.



REMEMBER

No secrets in code

No secrets in Git

No excessive permissions

No shared credentials

No unmanaged configs

No drift in environments



@veriqta



veriqta



veriqta



veriqta



@veriqta

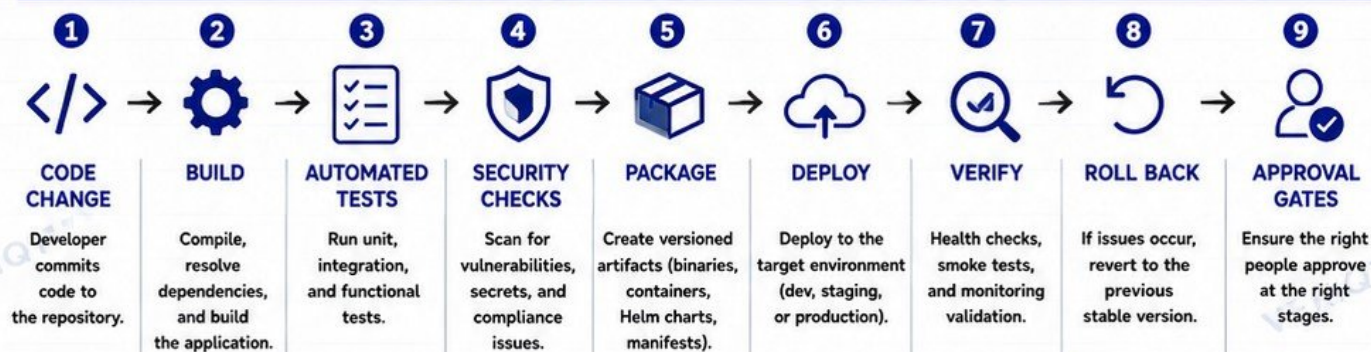


@veriqta

CI/CD FOR CUSTOMER DEPLOYMENTS

AUTOMATE. VALIDATE. DEPLOY. VERIFY. DELIVER VALUE.

THE CI/CD PIPELINE FOR CUSTOMER DEPLOYMENTS



DEPLOYMENT ARTIFACTS

Artifacts are the versioned outputs of the pipeline. They must be immutable, traceable, and repeatable.

- CONTAINER IMAGES**
Docker images stored in a registry.
- HELM CHARTS**
Packaged Kubernetes applications.
- MANIFESTS**
Kubernetes YAML or other deployment files.
- CONFIGURATION PACKS**
Environment-specific configuration files.
- BUILD REPORTS**
Test results, coverage, and quality reports.
- SBOM & SECURITY REPORTS**
Software Bill of Materials and scan results.
- VERSION TAGS**
Semantic versions for traceability.

APPROVAL GATES EXPLAINED

- AUTOMATED GATES**
Tests pass, security checks clear, quality thresholds met.
- MANUAL GATES**
Human approval for risky or production changes.
- CHANGE ADVISORY**
Required for high-impact or regulated environments.
- TIME WINDOWS**
Deployments allowed only during approved maintenance windows.

WHY GATES MATTER

- ✓ Prevent bad code from reaching customers
- ✓ Ensure compliance and security
- ✓ Reduce risk in production
- ✓ Give confidence in every release

WHY REPEATABLE DEPLOYMENTS MATTER FOR FDE TEAMS

- ✓ **CONSISTENCY**
Same process every time reduces human error.
- ✓ **SPEED**
Automated pipelines deliver faster and safely.
- ✓ **RELIABILITY**
Customers get stable, predictable releases.
- ✓ **TRACEABILITY**
Every deployment is versioned and auditable.
- ✓ **RECOVERY**
Easy rollbacks minimize impact when things go wrong.
- ✓ **TRUST**
Customers trust systems that are delivered reliably.
- ✓ **FOCUS**
FDEs spend time solving problems, not repeating manual tasks.

CI/CD BEST PRACTICES

- ✓ Automate everything you can.
- ✓ Fail fast and fix early.
- ✓ Keep pipelines simple and observable.
- ✓ Use versioned, immutable artifacts.
- ✓ Test in environments that mirror production.
- ✓ Never bypass security or quality gates.
- ✓ Monitor after every deployment.

WHEN THE PIPELINE FAILS



REMEMBER

A great CI/CD pipeline delivers value safely and repeatedly. Your customer's success depends on your ability to ship reliable, secure, and repeatable deployments.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

OBSERVABILITY AND PRODUCTION DEBUGGING

SEE EVERYTHING. UNDERSTAND FAST. FIX THE RIGHT THING.

THE 5 PILLARS OF OBSERVABILITY



METRICS

- Numerical signals
- Trends over time
- System health
- Capacity, latency, errors, saturation
- Answer: How is the system performing?



LOGS

- Discrete events
- What happened?
- Structured or unstructured
- Detailed context
- Answer: What happened?



TRACES

- Request journey across services
- End-to-end visibility
- Latency breakdown
- Answer: Where is the problem?



ALERTS

- Proactive notifications
- Based on thresholds or anomalies
- Reduce MTTR
- Answer: When does something need attention?



DASHBOARDS

- Visualize key signals
- Real-time overview
- Share insights
- Drive decisions
- Answer: What is the big picture?



CORRELATION IDS

- Unique ID for each request or transaction
- Pass through logs, traces, and services
- Connect the dots across distributed systems
- Essential for fast debugging

APPLICATION vs INFRASTRUCTURE FAILURES



APPLICATION

- Code bugs
- Logic errors
- Exceptions
- Bad data



INFRASTRUCTURE

- CPU/Memory
- Network
- Disk/Storage
- Third-party dependencies

ESTABLISHING A TIMELINE



- Know when the issue started
- Align events across systems
- Use metrics, logs, traces, deployments, and changes
- Build a clear sequence
- Context is everything

ASKING "WHAT CHANGED?"



- Code deploys?
- Config changes?
- Infra changes?
- Traffic spikes?
- Dependency updates?
- Certificates/Secrets?
- Always compare before diving deep



EVIDENCE BEFORE ASSUMPTIONS

Don't guess. Don't assume. Don't jump to conclusions. Collect evidence, then build your theory. Let the data tell you the story.



AVOID THESE TRAPS

- Blaming the wrong layer
- Ignoring recent changes
- Focusing on symptoms only
- Overlooking dependencies
- Fixing without verifying

THE DEBUGGING WORKFLOW: FOLLOW THE EVIDENCE



1. SYMPTOM

What is the user seeing? Define the problem clearly.



2. EVIDENCE

Collect data: metrics, logs, traces, alerts, events.



3. HYPOTHESIS

Form a theory about what might be causing it.



4. TEST

Run tests, queries, or experiments to validate.



5. CAUSE

Identify the root cause, not just the symptom.



6. FIX

Apply the right fix in the right place.



7. VERIFY

Confirm the issue is resolved. Monitor to make sure it stays fixed.



PRACTICAL CHECKLIST

- ✓ Reproduce or confirm the issue
- ✓ Check dashboards and alerts
- ✓ Review logs and traces with correlation IDs
- ✓ Check recent deployments and changes
- ✓ Verify infrastructure health
- ✓ Check dependencies and external services
- ✓ Narrow down to the failing component
- ✓ Apply fix or rollback
- ✓ Verify and monitor
- ✓ Document what happened



KEY REMINDERS

- ★ Observability is not just tools, it's a mindset.
- ★ You can't fix what you can't see.
- ★ High signal, low noise.
- ★ Good logs and metrics save hours.
- ★ Automation helps, but critical thinking wins.
- ★ Every incident is a chance to improve.



DEBUGGING INTEGRATION FAILURES

FIND THE ROOT CAUSE. FIX THE RIGHT LAYER.

COMMON INTEGRATION FAILURES AND WHAT THEY MEAN



CONNECTION REFUSED

WHAT IT MEANS:
The target service is not accepting connections.

CHECK:

- Service is running
- Correct host and port
- Firewall / security groups

TIPS:

nc -vz host port
curl -v http://host:port



DNS FAILURES

WHAT IT MEANS:
The domain name cannot be resolved.

CHECK:

- Correct DNS record
- DNS server reachability
- Spelling / TTL issues

TIPS:

nslookup domain.com
dig domain.com



TLS ERRORS

WHAT IT MEANS:
SSL/TLS handshake failed.

CHECK:

- Certificate is valid
- CA trusted
- Protocol / cipher mismatch

TIPS:

openssl s_client -connect host:443 -servername host



401 UNAUTHORIZED

WHAT IT MEANS:
Authentication is missing or invalid.

CHECK:

- Credentials / token
- Expired token
- Auth header / format

TIPS:

Check Authorization header and token validity



403 FORBIDDEN

WHAT IT MEANS:
Authenticated, but not allowed to access.

CHECK:

- Permissions / scopes
- Role / policy
- Resource ownership

TIPS:

Review IAM roles and access policies



404 NOT FOUND

WHAT IT MEANS:
The requested resource does not exist.

CHECK:

- Correct URL / path
- Resource exists
- Case sensitivity

TIPS:

Verify endpoint path and resource ID



429 TOO MANY REQUESTS

WHAT IT MEANS:
You are being rate limited.

CHECK:

- Rate limit policy
- Burst vs sustained rate
- Retry-After header

TIPS:

Implement retries with exponential backoff



500 SERVER ERRORS

WHAT IT MEANS:
The server encountered an error.

CHECK:

- Server logs
- Recent deployments
- Dependencies

TIPS:

Check server logs and correlation ID



TIMEOUTS

WHAT IT MEANS:
The service did not respond in time.

CHECK:

- Network latency
- Service performance
- Timeout settings

TIPS:

Increase timeout and check performance



INVALID PAYLOADS

WHAT IT MEANS:
The request body is invalid or malformed.

CHECK:

- JSON / schema
- Required fields
- Data types

TIPS:

Validate payload before sending. Log requests.



DEPENDENCY FAILURES

WHAT IT MEANS:
A downstream service or resource is failing.

CHECK:

- Dependency health
- Circuit breakers
- Timeouts / retries

TIPS:

Test dependency directly. Isolate the failing link.



GENERAL TIP

- ✓ Don't guess. Gather evidence.
- ✓ Check one layer at a time.
- ✓ Reproduce the issue.
- ✓ Use correlation IDs.
- ✓ Check recent changes.
- ✓ Look at logs, metrics, and traces together.
- ✓ Communicate clearly with your team.

DEBUGGING FROM CLIENT → NETWORK → SERVICE → DEPENDENCY



1. CLIENT

request correct?

- Headers, auth, payload
- Use curl / Postman
- Check client logs

TOOLS: curl, Postman, Browser DevTools



2. NETWORK

- Can client reach service?
- DNS, routing, firewall
- Latency or packet loss?
- TLS handshake working?

TOOLS: ping, traceroute, nslookup, openssl, tcpdump



3. SERVICE

- Is the service healthy?
- Check logs and metrics
- Correct config & ports?
- Auth and permissions?

TOOLS: kubectl, logs, metrics, config, APM



4. DEPENDENCY

- Are downstream services up?
- DB, queues, cache, APIs
- Timeouts or errors?
- Rate limits?

TOOLS: direct tests, logs, health checks



QUICK DEBUG CHECKLIST

- ✓ Reproduce the issue consistently
- ✓ Collect logs, metrics, and traces
- ✓ Identify the failing layer
- ✓ Form a hypothesis
- ✓ Test and confirm the root cause
- ✓ Implement the fix
- ✓ Verify the solution end-to-end
- ✓ Document what happened



REMEMBER

Every integration problem has a root cause. Go layer by layer. Use evidence. Fix the real problem, not just the symptom.



GOLDEN RULE

You don't fix what you don't understand. Observe. Investigate. Verify. Then fix.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SECURITY IN CUSTOMER ENVIRONMENTS

SECURE BY DESIGN. PROTECT DATA. EARN TRUST.



1. LEAST PRIVILEGE

- Give users, apps, and services only the minimum access they need.
- Regularly review and remove unnecessary permissions.



2. IAM (IDENTITY AND ACCESS MANAGEMENT)

- Centralize identity management.
- Use roles, groups, and policies.
- Enforce MFA for all human users.



3. AUTHENTICATION VS AUTHORIZATION

- Authentication: Proving who you are.
- Authorization: Determining what you can do.
- Never confuse the two.



4. SECRET PROTECTION

- Store secrets in dedicated secret managers.
- Rotate secrets regularly.
- Never store secrets in code or configs.



5. ENCRYPTION

- Encrypt data in transit (TLS).
- Encrypt data at rest.
- Manage keys securely and rotate them.



6. NETWORK BOUNDARIES

- Segment networks.
- Use firewalls and security groups.
- Allow only required inbound and outbound traffic.



7. AUDIT LOGS

- Enable and centralize audit logging.
- Monitor access, changes, and admin actions.
- Logs are your evidence.



8. CUSTOMER DATA PROTECTION

- Understand the data you handle.
- Minimize collection.
- Classify data.
- Comply with privacy regulations (GDPR, CCPA, etc.).



9. DEPENDENCY SECURITY

- Scan dependencies for vulnerabilities.
- Keep libraries and base images updated.
- Use trusted registries and sources.



10. PRODUCTION ACCESS

- Limit production access to only those who need it.
- Use just-in-time (JIT) access when possible.
- All access must be logged and monitored.



11. NEVER DISABLE SECURITY CONTROLS

Do not turn off security controls, TLS verification, firewalls, or auth checks just to make an integration work. Fix the root cause the right way.



SECURITY IS A TEAM SPORT

- Share risks early.
- Work with the customer's security team.
- Document decisions and exceptions.

SECURITY LAYERS: DEFENSE IN DEPTH



USERS & IAM
Who can access



NETWORK
Where traffic can flow



APPLICATION
How apps are built & configured



DATA
How data is protected



MONITORING
How we detect and respond



REMEMBER

Security is not optional. Shortcuts today become breaches tomorrow.
Secure systems. Protect customers. Build trust.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

DESIGNING CUSTOMER-SPECIFIC SOLUTIONS WITHOUT CREATING CHAOS

SOLVE TODAY'S PROBLEM WITHOUT DESTROYING TOMORROW'S PRODUCT.

1 PRODUCT CAPABILITY vs CUSTOMIZATION



- Product capability solves for many customers.
- Customization solves for one.
- Default to product. Customize only when absolutely necessary.
- Reusable > One-off.

2 CONFIGURATION BEFORE FORKS



- Use configuration, not code changes.
- Externalize settings.
- Make behavior flexible.
- Forks create long-term pain.

3 REUSABLE COMPONENTS



- Build components that can be reused across customers.
- Create extension points, not custom branches.
- Design for reuse from day one.

4 FEATURE FLAGS



- Ship code. Control behavior.
- Roll out safely.
- Test in production without risk.
- Enable / disable per tenant or environment.

5 INTEGRATION ADAPTERS



- Isolate external systems behind adapters.
- Keep core product clean.
- Easier to change, test, and replace.

6 TECHNICAL DEBT



- Every customization adds future cost.
- Track it.
- Pay it down.
- Don't borrow from the future without a plan.

7 UPGRADEABILITY



- Custom code breaks upgrades.
- Keep custom logic outside the core.
- Follow extension patterns.
- Test upgrades early.

8 MAINTAINABILITY



- Clear code.
- Clear boundaries.
- Clear ownership.
- Automated tests.
- Good documentation.
- Future you is a user.

9 DOCUMENTING DEVIATIONS



- Document what changed.
- Why it changed.
- How it works.
- What to watch.
- Make it easy to revisit.

10 KNOW WHEN A ONE-OFF SOLUTION SHOULD BECOME A PRODUCT CAPABILITY



Turn a one-off into a product capability when:

- ✓ 3 or more customers ask for it
- ✓ The solution is stable and proven
- ✓ It reduces support and complexity

- ✓ It aligns with product direction
- ✓ It saves engineering time long term

Don't just solve the problem. Improve the product for everyone.



JUNIOR ENGINEER TIP

A great FDE delivers value today without making the product harder to evolve tomorrow. Think long term. Build with discipline.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SHIPPING SAFELY INTO PRODUCTION

A DISCIPLINED DEPLOYMENT TODAY PREVENTS INCIDENTS TOMORROW.

1



PRE-DEPLOYMENT CHECKS

- Verify code build and tests passed
- Security scans completed
- Configuration validated
- All dependencies available
- Environments ready
- Access and permissions confirmed

2



CHANGE REVIEW

- Review what is changing and why
- Assess risk and impact
- Involve the right stakeholders
- Approve the change
- Document decisions

3



BACKUPS

- Take backups before deployment
- Verify backup integrity
- Store backups securely
- Know how to restore
- Test restores regularly

4



DATABASE MIGRATIONS

- Plan schema changes carefully
- Use versioned migrations
- Test in staging first
- Make migrations idempotent
- Have a rollback strategy

5



FEATURE FLAGS

- Hide new features until ready
- Enable gradually
- Roll back instantly
- Reduce deployment risk
- Control per environment / tenant

6



CANARY RELEASES

- Deploy to a small subset first
- Monitor for errors and latency
- Validate behavior
- Gradually increase traffic
- Roll back quickly if needed

7



ROLLBACK PLANNING

- Define rollback triggers
- Keep previous version ready
- Automate rollback steps
- Test rollback process
- Communicate rollback plan

8



DEPLOYMENT VERIFICATION

- Verify services are healthy
- Check logs and metrics
- Validate critical user flows
- Run smoke / integration tests
- Confirm external dependencies

9



MONITORING AFTER RELEASE

- Monitor metrics, logs, and alerts
- Watch error rates and latency
- Set alert thresholds
- Compare with baseline
- Stay in high-alert mode initially

10



CUSTOMER COMMUNICATION

- Inform customers before deployment
- Share expected impact or downtime
- Provide status updates
- Communicate when complete
- Share rollback if needed



**DEPLOYMENT IS NOT COMPLETE
UNTIL THE SYSTEM IS VERIFIED.**

SHIP WITH CONFIDENCE. VERIFY WITH DISCIPLINE.

DEPLOYMENT FLOW



THE FORWARD DEPLOYED ENGINEER INCIDENT

A DISCIPLINED RESPONSE. FAST ACTION. CLEAR OUTCOME.



INCIDENT RESPONSE MINDSET

-  **STAY CALM. FOCUS ON FACTS.**
-  **EVIDENCE BEFORE ASSUMPTIONS.**
-  **COMMUNICATE EARLY. COMMUNICATE CLEARLY.**
-  **OWN THE OUTCOME. NOT JUST THE FIX.**
-  **EVERY INCIDENT IS A CHANCE TO GET BETTER.**

KEEP THESE HANDY

- ✓ Monitoring Dashboards
- ✓ Log Search
- ✓ Runbooks
- ✓ Recent Change Log
- ✓ Rollback Plan
- ✓ Customer Contacts
- ✓ Status Page / Comms Channel

REMEMBER

-  **SPEED HELPS. DISCIPLINE SAVES. LEARNINGS SCALE.**

JUNIOR ENGINEER TIP

-  Incidents are not failures. They are feedback. Handle them like an engineer. Make the system stronger.

INCIDENT RESPONSE FLOW



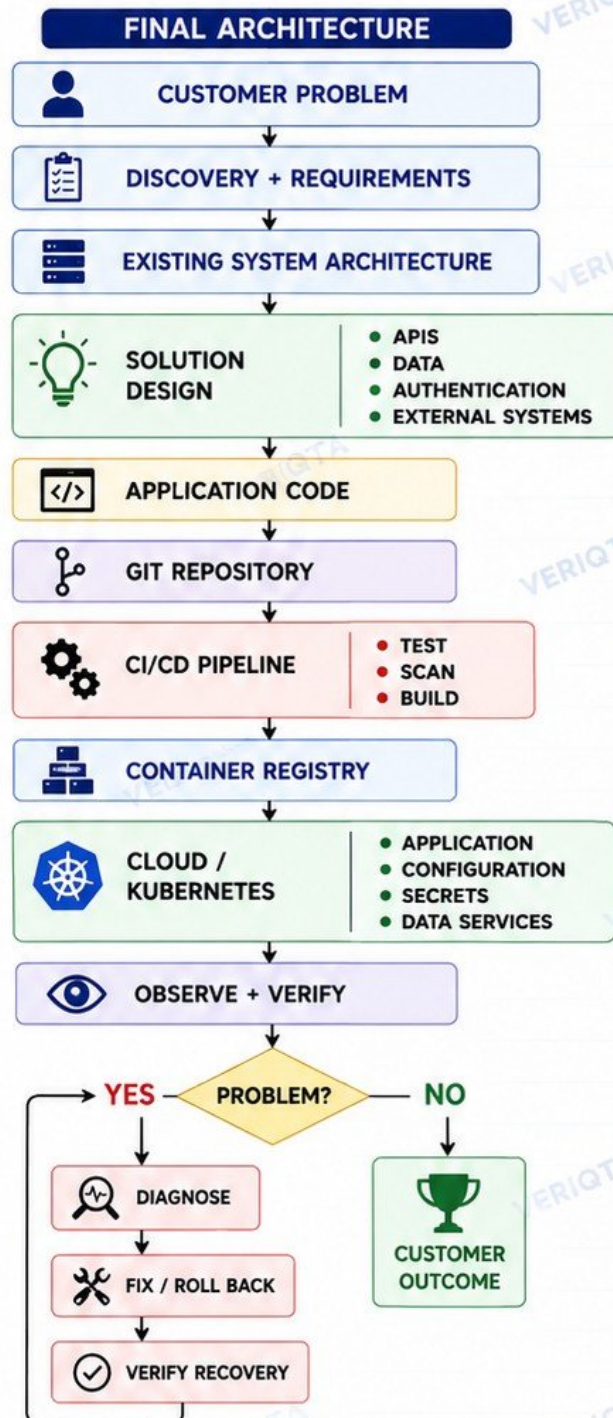
FINAL PRODUCTION MISSION

FROM CUSTOMER PROBLEM TO WORKING SYSTEM

- 1  RECEIVE AN AMBIGUOUS CUSTOMER REQUIREMENT
- 2  DISCOVER THE REAL PROBLEM
- 3  MAP THE CUSTOMER'S ARCHITECTURE
- 4  DEFINE SUCCESS CRITERIA
- 5  DESIGN THE INTEGRATION
- 6  BUILD THE APPLICATION COMPONENT
- 7  CONNECT AN EXTERNAL API
- 8  CONFIGURE AUTHENTICATION
- 9  CONTAINERIZE THE APPLICATION
- 10  CONFIGURE SECRETS SAFELY
- 11  DEPLOY TO KUBERNETES
- 12  EXPOSE THE SERVICE
- 13  ADD HEALTH CHECKS
- 14  ADD LOGGING AND METRICS
- 15  RUN INTEGRATION TESTS
- 16  DEPLOY THROUGH CI/CD
- 17  INTRODUCE A CONTROLLED FAILURE
- 18  DIAGNOSE IT FROM EVIDENCE
- 19  RESTORE SERVICE
- 20  VERIFY THE CUSTOMER OUTCOME



MISSION COMPLETE
THE CUSTOMER GETS VALUE.
THE SYSTEM IS RELIABLE.
YOU DELIVERED.



DISCOVER. DESIGN. BUILD. DEPLOY. OBSERVE. RECOVER. DELIVER VALUE.
THAT IS THE FORWARD DEPLOYED ENGINEER MISSION.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta